

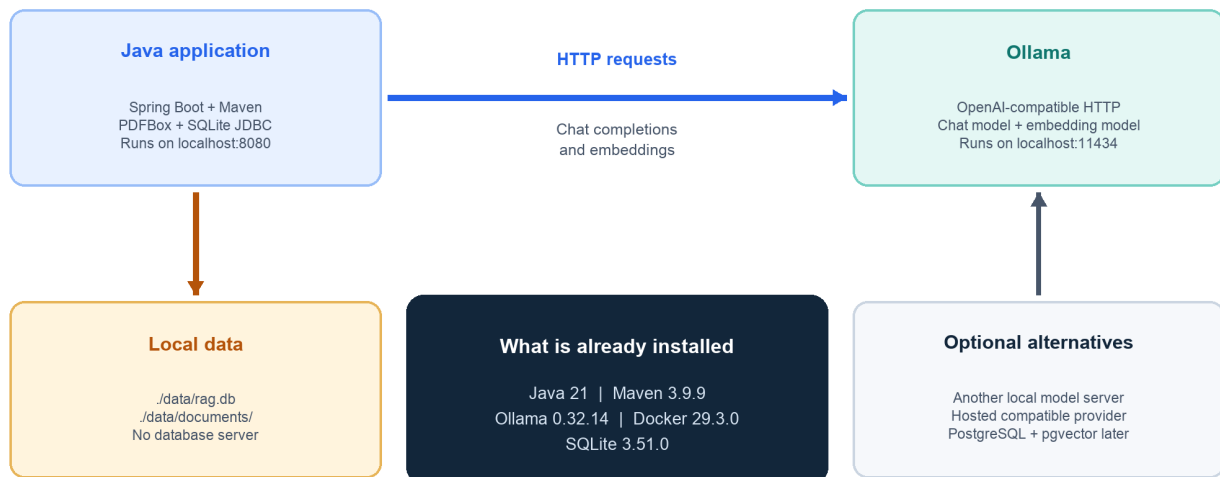
MINI RAG ENGINE

Project Plan

A local-first, provider-neutral document question-answering system
built in Java without LangChain

Local-first runtime

Default V1 requires no Oracle infrastructure, cloud account or database server



One remaining prerequisite for real RAG: pull an embedding model. No model download is needed for fake-provider tests.

Planning baseline | 18 August 2026

Executive decision

Proceed with Java 21, Spring Boot, Maven, PDFBox, embedded SQLite, exact cosine search in Java, and independently configurable OpenAI-compatible chat and embedding clients. Use Ollama locally by default and deterministic fake providers for automated tests. No Oracle infrastructure or cloud account is required.

Readiness

Ready now	Remaining before real end-to-end RAG
Java 21, Maven 3.9.9, running Ollama 0.32.14, Docker, SQLite	Pull an embedding model such as embeddinggemma; select a chat model; allow Maven to obtain dependencies on the first build

Document map

Sections 1-3 define the product and RAG flow. Sections 4-6 define external models, provider portability, and storage. Sections 7-9 specify architecture, APIs, and implementation phases. Sections 10-14 cover testing, acceptance, risks, learning outcomes, and the recommended decisions.

1. Project objective

Build a small document question-answering system in Java. A user uploads PDFs and asks questions. The system retrieves the most relevant passages and asks a language model to answer from those passages, returning source citations. We will implement text chunking, embedding orchestration, cosine similarity, ranking, context construction, and the RAG pipeline ourselves. We will not use LangChain or bind the application to Oracle infrastructure.

The project must be local-first, open-source friendly, simple to run, and provider-neutral. Changing from Ollama to another local or hosted OpenAI-compatible model server should normally be configuration, not a code rewrite.

2. What V1 will build

Area	V1 behavior
Document ingestion	Upload one or more text-based PDF files
Text extraction	Extract text and page boundaries with Apache PDFBox
Chunking	Create paragraph-aware chunks with configurable size and overlap
Embeddings	Generate an embedding for every document chunk and every question
Persistence	Store document metadata, chunks, and vectors in one local SQLite file
Retrieval	Calculate cosine similarity in Java and rank chunks ourselves
Answer generation	Send the question and only the selected context to a chat model
Grounding	Instruct the model to decline when the retrieved evidence is insufficient
Citations	Return filename, page, chunk index, rank, and similarity score
Inspection	Allow search without answer generation so retrieval can be debugged separately

V1 boundaries

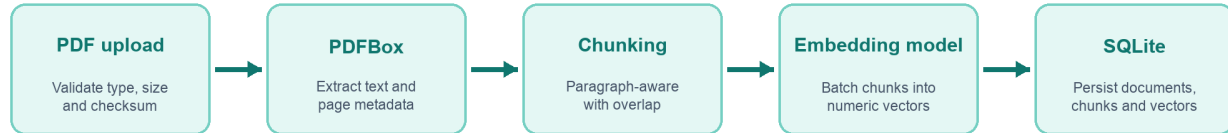
Include	Defer
PDFs with extractable text	OCR for scanned PDFs
REST API	Full web UI
One local user	Authentication and authorization
Exact vector scan in Java	Approximate nearest-neighbor index
Local SQLite persistence	Distributed or hosted database
Ollama and generic OpenAI-compatible clients	Provider-specific SDK collection
Synchronous answer API	Streaming responses
Evaluation question set	Automated LLM-as-judge evaluation

3. How RAG works

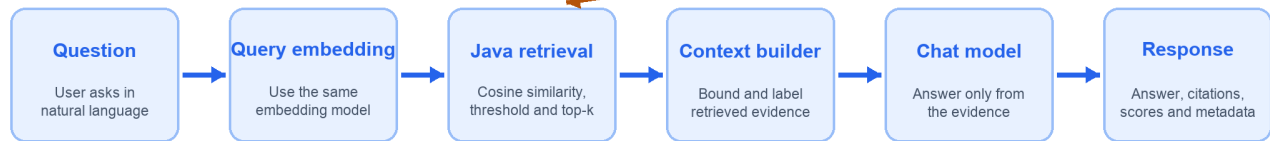
Mini RAG system architecture

Two controlled paths: index documents once, retrieve evidence for every question

INDEXING PATH



QUESTION PATH



Control boundary: the Java application owns files, database access, ranking and prompt construction. The chat model receives only the question and selected evidence - never direct database or filesystem access.

RAG has two separate paths.

Indexing path

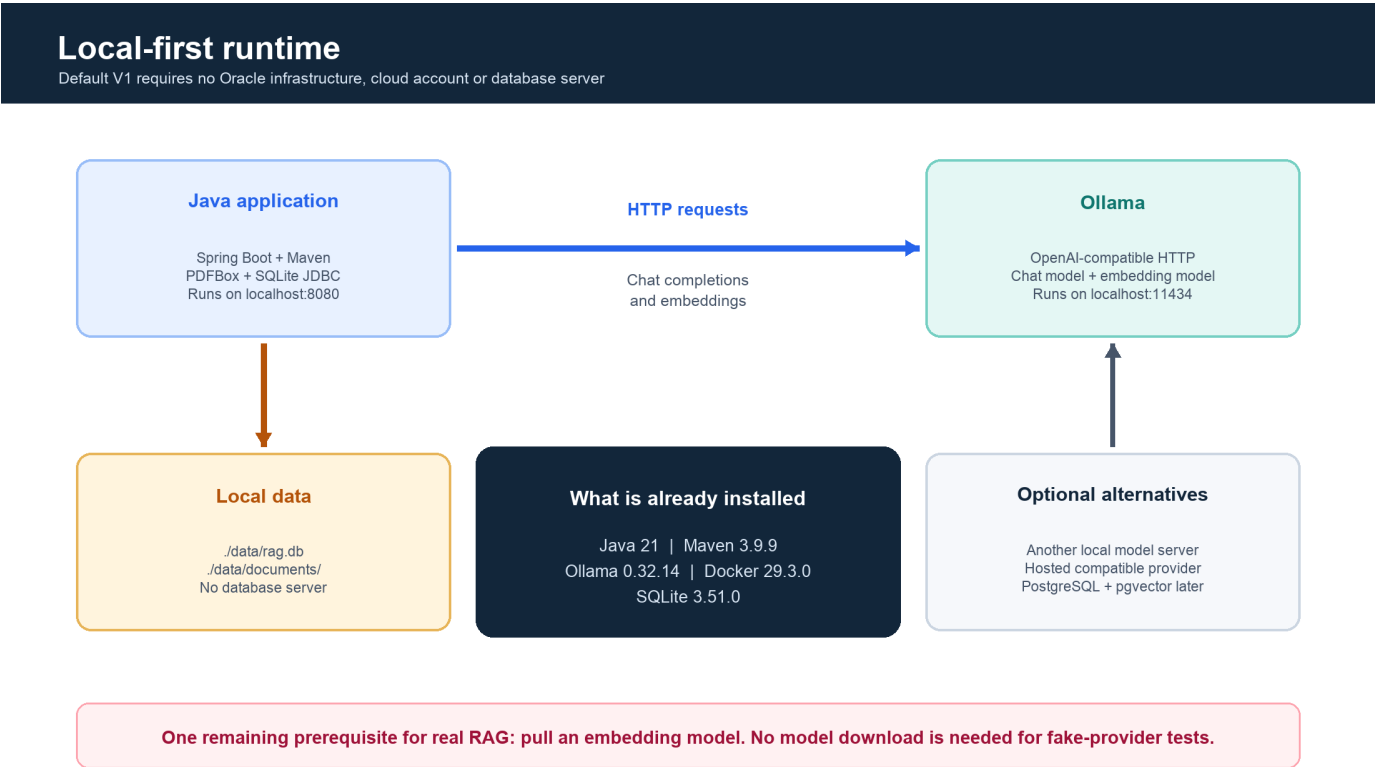
1. Accept and validate a PDF.
2. Calculate a checksum and prevent accidental duplicate indexing.
3. Extract text while preserving document and page metadata.
4. Split text into overlapping, paragraph-aware chunks.
5. Send chunks in batches to an embedding model.
6. Validate vector dimensions and finite numeric values.
7. Store documents, chunks, embeddings, model name, and dimensions in SQLite.

Question path

1. Accept a question.
2. Generate an embedding for the question using the same embedding model used for the indexed chunks.
3. Load candidate embeddings from SQLite.
4. Calculate cosine similarity in Java.
5. Rank results, apply a minimum score, and select top-k chunks.
6. Build a bounded prompt containing the question and labelled evidence.
7. Ask the chat model to answer only from that evidence.
8. Return the answer, citations, retrieved chunks, scores, and model metadata.

The chat model never receives direct database or filesystem access. The Java application controls the evidence it sees.

4. Models and external dependencies



RAG usually uses two model capabilities. They may be served by the same runtime but are independently configured.

Capability	Input	Output	Purpose
Embedding model	Chunk or question	Numeric vector	Semantic comparison and retrieval
Chat model	Question plus retrieved evidence	Human-readable answer	Grounded response generation

An ordinary chat LLM is not automatically an embedding model. If a custom local server provides chat only, the application can use a separate local embedding model.

Recommended local-first default

Layer	Default
Java runtime	Java 21
Build	Maven
Web application	Spring Boot
PDF extraction	Apache PDFBox
Model runtime	Ollama on localhost
Model protocol	OpenAI-compatible HTTP
Embedding model	embeddinggemma initially
Chat model	Configurable; start with a small model suitable for the machine
Database	SQLite through JDBC
Vector search	Exact scan and cosine similarity implemented in Java

Verified workstation status

Prerequisite	Status on 18 August 2026	Action
Java	Java 21.0.8 installed	None
Maven	Maven 3.9.9 installed	None
Ollama	Version 0.32.14 installed and API responding	None
Docker	Version 29.3.0 installed	Optional; not required for V1
SQLite command-line tool	Version 3.51.0 installed	Optional; application uses SQLite JDBC
Chat model	Several older local models already present	Usable, but pull a smaller current model if preferred
Embedding model	No embedding model found in <code>ollama list</code>	Pull one before the first real end-to-end test

No Oracle VPN, OCI tenancy, Oracle image, Kubernetes cluster, cloud database, or cloud API key is required.

Before the first real local RAG test:

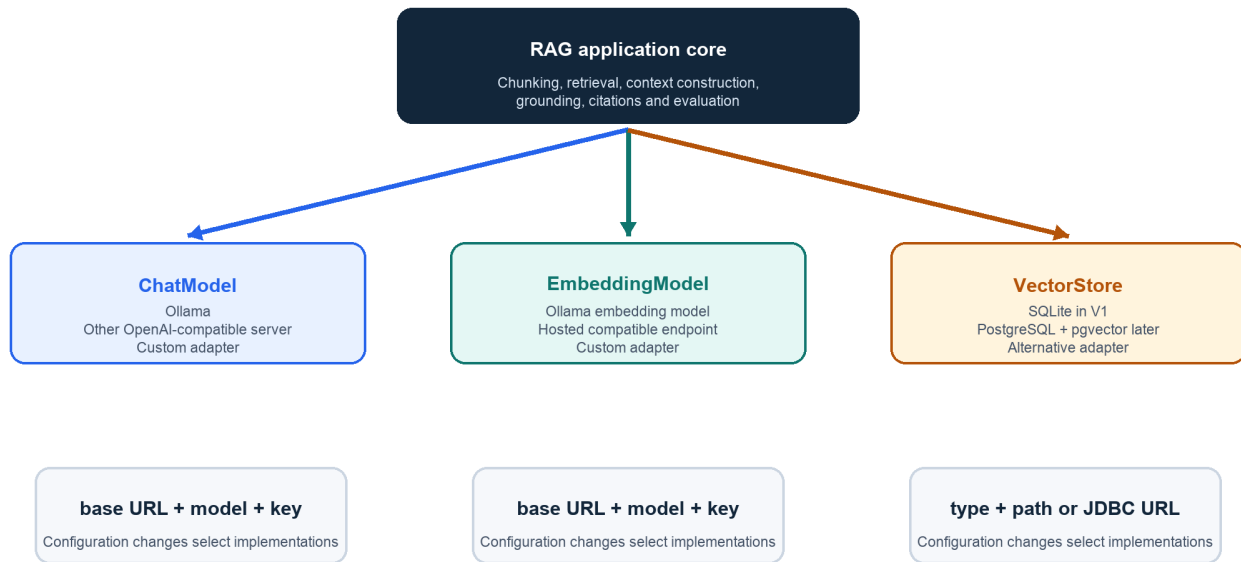
```
ollama pull embeddinggemma
# Select and pull a chat model only if none of the existing models is suitable.
```

The first Maven build will download open-source Java dependencies such as Spring Boot, PDFBox, Jackson, JUnit, and SQLite JDBC. After dependencies and models are cached, development and tests can run locally. Unit tests will use fake model implementations and will not require Ollama or internet access.

5. Plug-and-play model design

Provider-neutral design

Chat, embeddings and storage vary independently behind small Java interfaces



Embedding-model changes require re-indexing. Chat-model changes do not.

Use two small application interfaces:

```

public interface ChatModel {
    ChatResponse generate(ChatRequest request);
}

public interface EmbeddingModel {
    List<float[]> embed(List<String> inputs);
}

```

Initial adapters:

- `OpenAiCompatibleChatModel`
- `OpenAiCompatibleEmbeddingModel`
- `FakeChatModel` for automated tests
- `FakeEmbeddingModel` for automated tests

Example local configuration:

```

rag:
  chat:
    type: openai-compatible
    base-url: http://localhost:11434/v1
    model: llama3.2
    api-key: ollama
  embedding:
    type: openai-compatible
    base-url: http://localhost:11434/v1
    model: embeddinggemma
    api-key: ollama
  store:
    type: sqlite
    path: ./data/rag.db

```

Ollama ignores the placeholder API key. Cloud keys must come from environment variables and must never be committed.

Supported combinations

Available capability	Configuration
Local chat and local embeddings	Point both clients to Ollama or another local server
Local chat only	Local chat plus a separate local or hosted embedding endpoint
Local embeddings only	Local retrieval plus a hosted chat endpoint
Embeddings without chat	Semantic-search mode without answer generation
Non-compatible custom API	Add one adapter; leave the RAG pipeline unchanged

Changing the embedding model requires re-indexing because vectors from different models or dimensions are not comparable. Changing only the chat model does not require re-indexing.

6. Database plan

Use SQLite for V1. It is embedded, transactional, local, and requires no database server.

```
mini-rag-engine/  
  data/  
    rag.db  
    documents/
```

Proposed tables

Table	Important fields
documents	ID, original filename, checksum, media type, page count, created time
chunks	ID, document ID, page number, chunk index, text, token estimate
embeddings	Chunk ID, model, dimension, binary float vector, created time
index_jobs	Document ID, status, failure reason, timestamps

Store vectors as binary float32 values rather than verbose JSON. Validate dimensions when writing and reading them. For the small V1 dataset, load candidate vectors and scan them in Java. This keeps cosine similarity visible and testable.

Define a VectorStore interface so SQLite can later be replaced with PgVectorStore. PostgreSQL plus pgvector becomes useful when the dataset grows, concurrent writers matter, metadata filtering becomes complex, or approximate indexing is needed. It is not needed for V1.

7. Proposed architecture and packages

Package	Responsibility
api	REST controllers, request validation, response DTOs, exception mapping
document	Upload validation, checksums, PDF extraction, document lifecycle
chunking	Paragraph-aware splitting, overlap, page tracking
embedding	Provider interface, HTTP adapter, batching, dimensions
vector	Vector serialization, validation, cosine similarity
store	SQLite schema, repositories, vector-store abstraction
retrieval	Query embedding, exact ranking, threshold, top-k
generation	Evidence prompt and chat provider
rag	Orchestrate retrieval and generation
evaluation	Benchmark questions and retrieval metrics
config	Typed provider, model, storage, and limit settings

Core data types

Type	Fields
Document	ID, filename, checksum, page count, status
Chunk	ID, document ID, text, page, index
EmbeddedChunk	Chunk metadata, model, dimension, vector
SearchResult	Chunk, score, rank
RagAnswer	Answer, citations, retrieval details, model metadata

8. API plan

Method	Endpoint	Purpose
POST	/api/documents	Upload and index a PDF
GET	/api/documents	List indexed documents and states
GET	/api/documents/{id}	Inspect one document and indexing metadata
DELETE	/api/documents/{id}	Delete document, chunks, and vectors
POST	/api/search	Retrieve ranked chunks without calling the chat model
POST	/api/questions	Retrieve evidence and generate a cited answer
GET	/api/health	Report database and configured-provider readiness

/api/search is a first-class feature. It separates retrieval failures from generation failures.

9. Code plan

Phase	Implementation	Verification	Estimate
0. Scaffold	Maven module, Spring Boot, profiles, typed configuration	Application and health endpoint start	1-2 h
1. Vector math	Vector validation, serialization, cosine similarity	Deterministic unit tests and invalid-input tests	1-2 h
2. PDF ingestion	Upload limits, PDFBox extraction, checksum, page metadata	Known PDF text/page assertions	2-3 h
3. Chunking	Paragraph-aware chunks, configurable size and overlap	Boundary, empty-page, long-paragraph tests	2-3 h
4. Persistence	SQLite migrations and repositories	Restart persistence and cascade-delete tests	3-4 h
5. Embeddings	Interface, fake provider, OpenAI-compatible HTTP adapter, batches	Contract tests, timeout and dimension checks	2-3 h
6. Retrieval	Query embedding, exact scan, ranking, threshold, top-k	Expected chunks rank in top three	2-3 h
7. Generation	Evidence prompt, refusal policy, citations	Supported and unsupported question tests	2-3 h
8. Evaluation	Curated questions, expected sources, hit-rate report	Repeatable benchmark output	2-3 h
9. Packaging	README, sample PDFs, scripts, optional container	Fresh-machine runbook test	2-3 h

Realistic learning-first duration: three to four focused days. A demo can appear earlier, but evaluation and failure analysis are essential project outcomes.

10. Testing strategy

Test level	External dependencies	Coverage
Unit	None	Chunking, cosine math, vector serialization, prompt construction
Repository integration	Temporary SQLite only	Migrations, persistence, deletion, restart
Provider contract	Local stub HTTP server	Payloads, parsing, authentication header, errors, timeouts
Pipeline integration	Fake deterministic models	Upload-to-answer behavior without internet or Ollama
Local model integration	Ollama	Real embeddings, retrieval quality, grounded answer behavior
Evaluation	Ollama plus small PDF set	Top-k hit rate, citation correctness, unsupported questions

Create at least 20 evaluation questions with expected source pages. Include questions that use different wording than the source, ambiguous questions, and questions not answered by any document.

11. Acceptance criteria

Requirement	Success condition
Easy startup	Documented local setup; no Oracle or cloud dependency
Persistence	Indexed documents survive application restart
Semantic retrieval	Relevant passages can be found without exact keyword matches
Correct math	Known vectors produce deterministic cosine rankings
Provider portability	Base URL and model can be changed without pipeline changes
Offline tests	Default automated suite passes without Ollama or internet
Grounding	Insufficient evidence produces an explicit refusal
Citations	Supported answers include document and page references
Inspectability	Search results and scores are available without invoking chat
Evaluation	At least 20 questions have expected evidence and a repeatable report

12. Engineering considerations before coding

Concern	Decision or mitigation
Prompt grounding is imperfect	Use thresholds, labelled evidence, citations, refusal instructions, and evaluation
Scanned PDFs	Detect little/no extracted text and return a clear V1 limitation; defer OCR
Prompt injection in documents	Treat retrieved text as untrusted evidence, never as system instructions
Model changes	Record embedding model and dimension; require re-indexing after embedding changes
Context limits	Bound top-k, chunk length, and total evidence size
Duplicate documents	Use a cryptographic content checksum
Large uploads	Limit file size, page count, extracted characters, and batch size
Model latency	Set connect/read timeouts; retry only transient failures
Secrets	Environment variables only; redact authorization headers and prompts from logs
Numerical safety	Reject mismatched dimensions, empty vectors, NaN, and infinity
Data deletion	Delete document, chunks, and vectors transactionally
Reproducibility	Pin dependency and container versions; do not use floating latest tags in automation
Observability	Log request IDs, stage durations, chunk counts, and provider latency without document content

13. What the project teaches

Concept	Implementation experience	Explanation goal
Embeddings	Convert chunks and questions to vectors	Explain semantic representation and model-specific vector spaces
Chunking	Preserve useful context with overlap	Explain recall/precision and chunk-size trade-offs
Similarity	Implement cosine similarity	Explain dot product, magnitude, normalization, and ranking
Retrieval	Threshold and rank candidates	Explain top-k, false positives, and retrieval debugging
Context engineering	Build a bounded evidence prompt	Explain token budgets and evidence ordering
Grounding	Refuse unsupported answers	Explain why prompts reduce but do not eliminate hallucinations
Persistence	Store chunks and vectors	Explain re-indexing and embedding compatibility
Evaluation	Curate expected evidence	Explain retrieval hit rate separately from answer quality
Architecture	Swap providers and stores	Explain dependency inversion and adapter boundaries

14. Recommended decisions

Proceed with Java 21, Spring Boot, Maven, PDFBox, SQLite, exact cosine search in Java, and two independently configurable OpenAI-compatible clients. Use local Ollama as the default runtime and fake providers for automated tests. Keep Docker optional and defer pgvector, OCR, authentication, UI, agents, and streaming until after V1 acceptance criteria pass.

15. References

- Ollama OpenAI compatibility: <https://docs.ollama.com/api/openai-compatibility>
- Ollama embeddings: <https://github.com/ollama/ollama/blob/main/docs/capabilities/embeddings.mdx>
- Ollama Docker: <https://docs.ollama.com/docker>
- SQLite overview: <https://sqlite.org/about.html>
- pgvector: <https://github.com/pgvector/pgvector>